

```
@login_required
def new_entry(request, topic_id):
    --snip--

@login_required
def edit_entry(request, entry_id):
    --snip--
```

Try accessing each of these pages while logged out; you should be redirected back to the login page. You'll also be unable to click links to pages such as `new_topic`. But if you enter the URL `http://localhost:8000/new_topic/`, you'll be redirected to the login page. You should restrict access to any URL that's publicly accessible and relates to private user data.

Connecting Data to Certain Users

Next, we need to connect the data to the user who submitted it. We only need to connect the data highest in the hierarchy to a user, and the lower-level data will follow. In Learning Log, topics are the highest level of data in the app, and all entries are connected to a topic. As long as each topic belongs to a specific user, we can trace the ownership of each entry in the database.

We'll modify the Topic model by adding a foreign key relationship to a user. We'll then have to migrate the database. Finally, we'll modify some of the views so they only show the data associated with the currently logged-in user.

Modifying the Topic Model

The modification to `models.py` is just two lines:

```
models.py from django.db import models
          from django.contrib.auth.models import User

class Topic(models.Model):
    """A topic the user is learning about."""
    Text = models.CharField(max_length=200)
    date_added = models.DateTimeField(auto_now_add=True)
    owner = models.ForeignKey(User, on_delete=models.CASCADE)

    def __str__(self):
        """Return a string representing the topic."""
        Return self.text

class Entry(models.Model):
    --snip--
```

We import the User model from `django.contrib.auth`. Then we add an owner field to Topic, which establishes a foreign key relationship to the User model. If a user is deleted, all the topics associated with that user will be deleted as well.